

1. Difference between JPA, Hibernate and Spring Data JPA

JPA	Hibernate	Spring Data JPA
Java Specification	ORM Framework	Framework built on top of JPA
Defines interfaces	Implements JPA interfaces	Simplifies database access
EntityManager	Generates SQL	Uses repositories
Vendor independent	Vendor implementation	Reduces boilerplate code

Application



Spring Data JPA



JPA



Hibernate



Database

2. Find a Country Based on Country Code

Entity

@Entity

```
public class Country {
```

```
    @Id
```

```
    private String code;
```

```
    private String name;
```

```
    // getters setters
```

```
}
```

Repository

```
public interface CountryRepository
```

```
    extends JpaRepository<Country,String>{
```

```
}
```

Service

@Service

```
public class CountryService {
```

```
    @Autowired
```

```
    CountryRepository repo;
```

```
    public Country getCountry(String code){
```

```
        return repo.findById(code).orElse(null);
```

```
    }
```

```
}
```

3. Add a New Country

@Service

```
public class CountryService {
```

```
    @Autowired
```

```
    CountryRepository repo;
```

```
    public Country addCountry(Country country){
```

```
        return repo.save(country);
```

```
    }
```

```
}
```

Controller

```
@PostMapping("/countries")
```

```
public Country addCountry(@RequestBody Country country){
```

```
    return service.addCountry(country);
```

```
}
```

4. Query Methods Feature

Spring Data JPA automatically creates SQL from method names.

Suppose the entity is

@Entity

```
public class Employee{
```

```
    @Id
```

```
    private int id;
```

```
    private String name;
```

```
    private String department;
```

```
    private double salary;
```

```
}
```

Repository

```
public interface EmployeeRepository
```

```
extends JpaRepository<Employee,Integer>{
```

```
    Employee findByName(String name);
```

```
    List<Employee> findByDepartment(String department);
```

```
    List<Employee> findBySalaryGreaterThan(double salary);
```

```
    List<Employee> findByNameContaining(String word);
```

```
List<Employee> findByDepartmentOrderBySalaryDesc(String dept);  
  
}
```

Generated SQL (conceptually)

findByName()

↓

SELECT * FROM employee

WHERE name=?

Common Query Methods

Method	Meaning
findByName	Equal
findByAgeGreaterThan	>
findByAgeLessThan	<
findBySalaryBetween	Between
findByNameContaining	LIKE %text%
findByDepartment	Equals
findByDepartmentAndSalaryGreaterThan	AND
findByNameOrDepartment	OR
findAllOrderBySalaryDesc	ORDER BY

5. O/R Mapping (Object Relational Mapping)

ORM maps Java objects to database tables.

Example

@Entity

```
public class Employee{
```

```
    @Id
```

```
    private int id;
```

```
    private String name;
```

```
}
```

Table

EMPLOYEE

ID

NAME

Annotations

Annotation	Purpose
@Entity	Makes class a table
@Table	Table name
@Id	Primary key
@Column	Column mapping
@GeneratedValue	Auto Increment

Relationships

One-to-One

@OneToOne

```
private Address address;
```

One-to-Many

@OneToMany(mappedBy="department")

```
private List<Employee> employees;
```

Many-to-One

@ManyToOne

```
private Department department;
```

Many-to-Many

@ManyToMany

```
private List<Course> courses;
```

These relationships are frequently asked in interviews.

6. Hibernate Query Language (HQL)

HQL uses entity names instead of table names.

Example

@Query("from Employee")

```
List<Employee> getEmployees();
```

Find by department

@Query("from Employee where department=?1")

```
List<Employee> findDept(String dept);
```

Find salary greater than

```
@Query("from Employee where salary>?1")
```

```
List<Employee> findSalary(double salary);
```

Parameterized Query

```
@Query("from Employee where name=:name")
```

```
Employee getEmployee(@Param("name") String name);
```

7. Native Query

Uses SQL instead of HQL.

```
@Query(value="select * from employee",  
nativeQuery=true)
```

```
List<Employee> getEmployees();
```

```
@Query(value="select * from employee where department=?1",  
nativeQuery=true)
```

```
List<Employee> getDepartment(String dept);
```

Difference

HQL	Native Query
Uses Entity name	Uses Table name
Database Independent	Database Specific
Object Oriented	SQL

8. Spring REST using Spring Boot

A REST API allows clients to communicate over HTTP.

Flow

Browser



Controller



Service



Repository



Database

Create Spring Web Project

Dependencies

Spring Web

Spring Data JPA

MySQL Driver

Lombok (optional)

Main Class

@SpringBootApplication

public class DemoApplication {

 public static void main(String[] args){

 SpringApplication.run(DemoApplication.class,args);

 }

}

REST Controller

@RestController

@RequestMapping("/countries")

public class CountryController {

 @Autowired

}

GET Request

}

GET by ID

}

POST

}

PUT

```
@PutMapping("/{code}")
```

```
country.setCode(code);  
return service.addCountry(country);  
}
```

DELETE

```
@DeleteMapping("/{code}")  
public void deleteCountry(@PathVariable String code){  
    service.deleteCountry(code);  
}
```

HTTP Methods

Method Purpose

GET	Read
POST	Insert
PUT	Update
DELETE	Delete

Common REST Annotations

Annotation	Purpose
@RestController	REST controller
@RequestMapping	Base URL
@GetMapping	GET API
@PostMapping	POST API

Annotation	Purpose
@PutMapping	PUT API
@DeleteMapping	DELETE API
@RequestBody	Read JSON request
@PathVariable	URL parameter
@RequestParam	Query parameter
@Autowired	Dependency Injection